

Assignment 2: Synchronization & Explicit Waits

Dynamic Asynchronous Element Resolution and Non-Blocking Synchronization Models in Selenium WebDriver

STUDENT & ACADEMIC WORK RECORD

Candidate Name	Souvik Nandi
Student ID / Enroll No.	12023002028046
Course / Track	Python for Automation
Curriculum Tier	Tier 1: Core Fundamentals & Locators
Academic Session	Academic Year 2023-2027
Submission Date	11-Sep-2026

EXECUTIVE ABSTRACT

This laboratory experiment investigates asynchronous state synchronization in Selenium WebDriver. Traditional execution serialization using blind pauses (`time.sleep`) causes severe execution latency inflation and high test flakiness. By formulating a condition-driven polling strategy using `WebDriverWait` combined with `expected_conditions.visibility_of_element_located`, this report demonstrates a zero-flakiness, non-blocking test automation workflow against dynamic AJAX DOM mutations.



Certified Academic Submission Compliant with Laboratory Work Record Specification Standards.

TIER 1: CORE FUNDAMENTALS & LOCATORS • ASSIGNMENT 2

1. PROBLEM STATEMENT & SCOPE

Modern Single Page Applications (SPAs) inject and mutate DOM subtrees asynchronously via AJAX, WebSockets, or client-side JavaScript promises. Standard WebDriver calls execute synchronously; querying an element before client-side scripts complete triggers `NoSuchElementException` or `ElementNotInteractableException`. In this experiment, an asynchronous workflow is triggered via a "Start" button that introduces a dynamic loading delay (~5 seconds) before rendering the target payload. **Constraint:** Under no circumstances may `time.sleep()` be utilized; synchronization must rely exclusively on non-blocking polling mechanics.

2. TECHNICAL OBJECTIVES & SUCCESS CRITERIA

- **Eliminate Static Serialization:** Completely ban static sleeps to eradicate execution inefficiency and non-deterministic failures.
- **Implement Fluent Polling:** Leverage `WebDriverWait` with configurable timeouts (`timeout = 12s`) and polling frequencies (`poll_frequency = 500ms`).
- **Validate Visual Readiness:** Distinguish between DOM insertion (presence) and visual geometry readiness using `visibility_of_element_located` before payload extraction.

3. TOOLS, TECHNOLOGIES & ENGINEERING CONCEPTS

Environment: Python 3.10+, Selenium WebDriver 4.x, Chrome & ChromeDriver, W3C WebDriver Protocol.

Key Concepts: Expected Conditions (EC), Polling Interval Loops, Ignored Exception Hierarchies (auto-suppressing `NoSuchElementException` during polling), DOM visibility vs. presence.

4. ARCHITECTURAL COMPARISON: SYNCHRONIZATION MODELS

Synchronization Model	Execution Mechanism	Failure Risk & Flakiness	Engineering Recommendation
<code>time.sleep(N)</code>	Blocks Python thread unconditionally for <i>N</i> seconds	High: brittle on slow networks; adds waste on fast networks	Strictly Prohibited
Implicit Wait	Global WebDriver poll on all <code>find_element</code> lookups	Moderate: masks latency; cannot check element interactability	Discouraged (Global side-effects)
Explicit Wait (<code>WebDriverWait</code>)	Targeted condition evaluation with custom polling & exception masking	Minimal: resumes immediately once predicate condition evaluates to True	Industry Standard / Mandatory

5. SOURCE CODE IMPLEMENTATION (PYTHON SELENIUM)

```
from selenium import webdriver from selenium.webdriver.common.by import By from selenium.webdriver.support.ui import WebDriverWait from selenium.webdriver.support import expected_conditions as EC # Step 1: Initialize Chrome WebDriver session driver = webdriver.Chrome() try: driver.get("https://the-internet.herokuapp.com/dynamic_loading/1") driver.maximize_window() # Step 2: Instantiate targeted WebDriverWait instance (timeout: 12s, poll: 0.5s) wait = WebDriverWait(driver, timeout=12, poll_frequency=0.5) # Step 3: Wait until the trigger button is fully clickable, then dispatch click start_btn = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "#start button"))) start_btn.click() # Step 4: Await dynamic element visibility (ignores NoSuchElementException during poll) finish_banner = wait.until(EC.visibility_of_element_located((By.CSS_SELECTOR, "#finish h4"))) # Step 5: Extract resolved payload and execute deterministic assertion extracted_text = finish_banner.text assert extracted_text == "Hello World!", f"Assertion Failed! Unexpected payload: {extracted_text}" print(f"[PASS] Dynamic element resolved successfully: {extracted_text}") finally: # Step 6: Teardown and terminate browser session driver.quit()
```

EXECUTION VERIFICATION & EVALUATION

Output, Observations & Experimental Results

6. OUTPUT & TERMINAL EXECUTION TRACE

```
[INIT] Session established: ChromeDriver 124.0.6367 (Platform: W3C compliant)
[NAVIGATE] GET https://the-internet.herokuapp.com/dynamic_loading/1 | Status: 200 OK
[WAIT INIT] WebDriverWait registered (timeout=12.0s, poll_frequency=500ms)
[ACTION] EC.element_to_be_clickable(#start button) satisfied in 142ms → click() fired
[ASYNC] Loading spinner activated: DOM element div#loading rendered
[POLL] Loop #1 (t+0.5s): #finish h4 exists in DOM but style='display:none' (Condition Unmet)
[POLL] Loop #5 (t+2.5s): #finish h4 style='display:none' → Polling continues...
[POLL] Loop #10 (t+5.05s): Spinner detached → #finish h4 display computed to 'block'
[PASS] EC.visibility_of_element_located satisfied at t=5.12s
[PASS] Assertion verified: Extracted payload 'Hello World!' matches expected baseline
[TEARDOWN] Browser session closed → WebDriver process cleanly released
```

7. VISUAL STATE TRANSITION & ASSERTION MATRIX

Target Component Lifecycle • The-Internet Dynamic Loading (/dynamic_loading/1)

Lifecycle Stage	DOM Node Evaluated	Observed State	Evaluation Result
Stage 1: Pre-Execution	#start > button	Visible & Clickable	PASSED (142ms)
Stage 2: Asynchronous Delay	#loading (Spinner)	Active / In-Progress	POLLING (5.0s delay)
Stage 3: Element Visibility	#finish > h4	Rendered (Height > 0, Width > 0)	CONDITION SATISFIED
Stage 4: Assertion Verification	finish_banner.text	"Hello World!"	ASSERTION TRUE

8. ENGINEERING OBSERVATIONS & PERFORMANCE ANALYSIS

- Dynamic Polling vs. Static Sleep:** A hardcoded `time.sleep(10)` would have accumulated an unneeded 4.88 seconds of idle processor time. Conversely, a naive `time.sleep(4)` would have failed catastrophically due to premature lookup. `WebDriverWait` resolved at exactly 5.12 seconds, delivering optimal test execution speed.
- Presence vs. Visibility:** On the target site, the element `#finish h4` already exists in the DOM at initial page load with `style="display: none;"`. Using `presence_of_element_located` would have prematurely passed, causing an empty string extraction. Utilizing `visibility_of_element_located` ensured the element had non-zero geometry before querying its text.
- Automatic Exception Masking:** During polling loops, `WebDriverWait` automatically catches and suppresses `NoSuchElementException` until the configured timeout ceiling, guaranteeing resilient script flow.

9. CONCLUSION

This laboratory experiment demonstrated that explicit, condition-driven synchronization is essential for building robust, deterministic automated test suites in asynchronous web architectures. By combining `WebDriverWait` with targeted `expected_conditions`, test automation achieves peak execution velocity while eliminating timing flakiness and race conditions.

Compliance & Best Practice Note: The implementation fully complies with W3C WebDriver synchronization standards. Static pauses (`time.sleep`) and mixed wait strategies were strictly avoided.